

Environment Check!

Did you do environment setup?

» Yes?

- » `docker compose start`
- » `docker compose exec workshop bash`
- » Attach VS code to your running container
- » Run test commands

» No?



[https://github.com/b-robotized/
roscon2026_motion_planning_workshop](https://github.com/b-robotized/roscon2026_motion_planning_workshop)



<https://www.menti.com/al28cptk37mp>

Motion Planning Fundamentals

with Moveit[®]

Yara Shahin
Timotej Gašpar
Jennifer Bühler
Dr. Denis

ROSCon 2026 Workshop, Toronto 22.09.2026

“ We believe that **robotics** is the key to greater **prosperity** and **quality of life** around the world.

ROS 2 makes it accessible!

We are deploying it in real-world applications!

\$whoarewe

b»robotizerJ



\$whoarewe

- » Yara
- » Robotics developer
- » Control, Manipulation
- » Behaviors



- » Dr. Jennifer
- » Senior Robotacist
- » Project Lead, Architectures
- » AI, Behaviors, Mobile
- » Manipulation, Control



- » Dr. Timo
- » Senior Robotacist
- » Projects Lead, Architectures
- » AI, Behaviors, Manipulation, Control

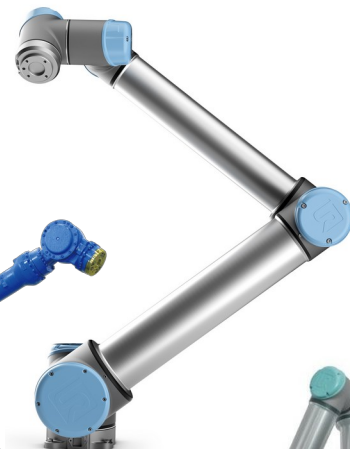
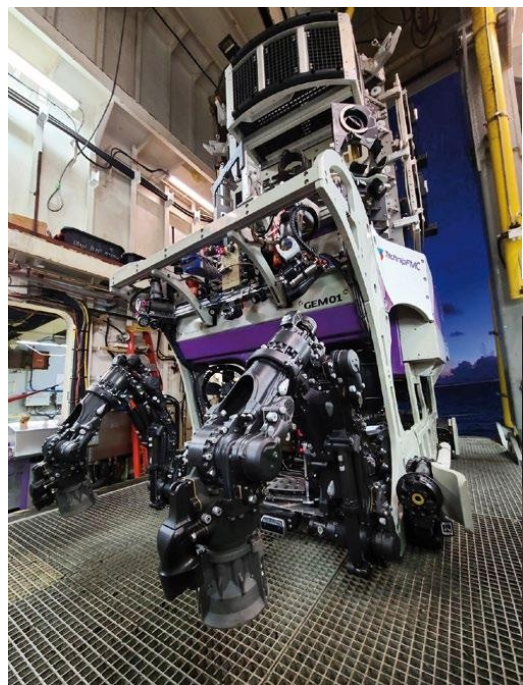


- » Dr. Denis
- » Owner and Senior Robocist
- » ros2_control Maintainer
- » Organizer ROSCon DE and HR



Examples ...

b»robotizerJ



By the end of this session

- ✓ How sampling-based planners work conceptually
- ✓ The architecture of Moveit 2: move group & planning scene
- ✓ How to configure your own robot
- ✓ Moveit servo
- ✓ Capstone project: design your own Pick & Place

Environment Check!

Did you do environment setup?

» Yes?

- » `docker compose start`
- » `docker compose exec workshop bash`
- » Attach VS code to your running container
- » Run test commands

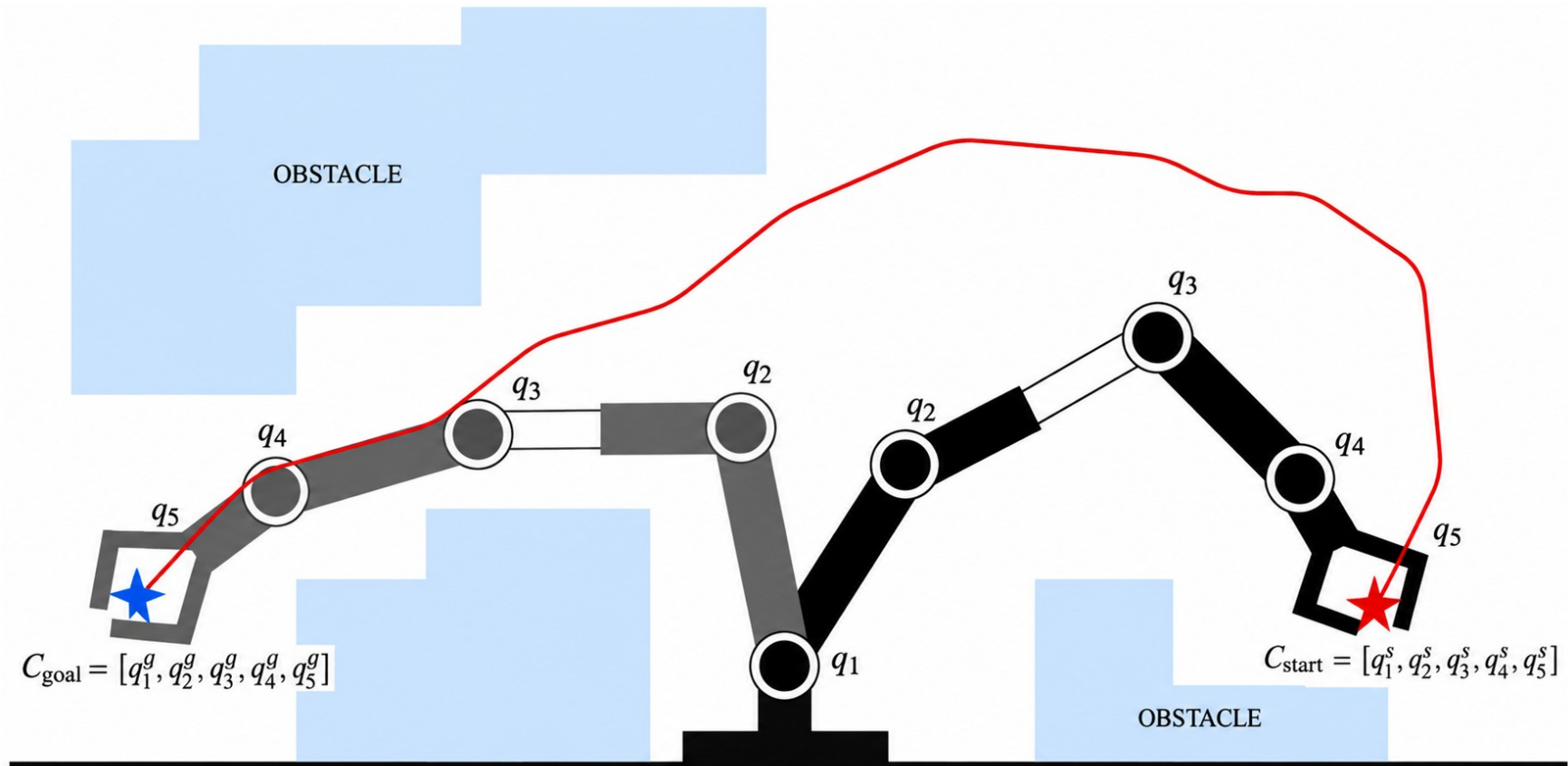
» No?



What is Motion Planning?

- » The Problem
- » Configuration Space
- » Forward and Inverse Kinematics
- » Constraints
- » Sampling-based planners

The Problem: Go from A to B

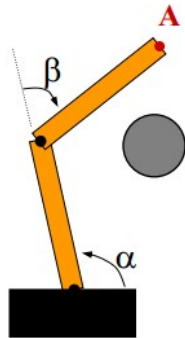


Problem Definition

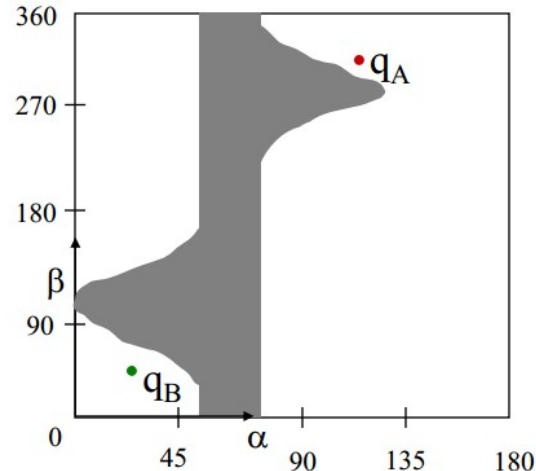
How to go from joint states A to joint states B?

Configuration Space

- » **All possible** combinations of joint positions.
- » Each point in the space is n joint positions.
- » For each possible variation: is this one in collision?



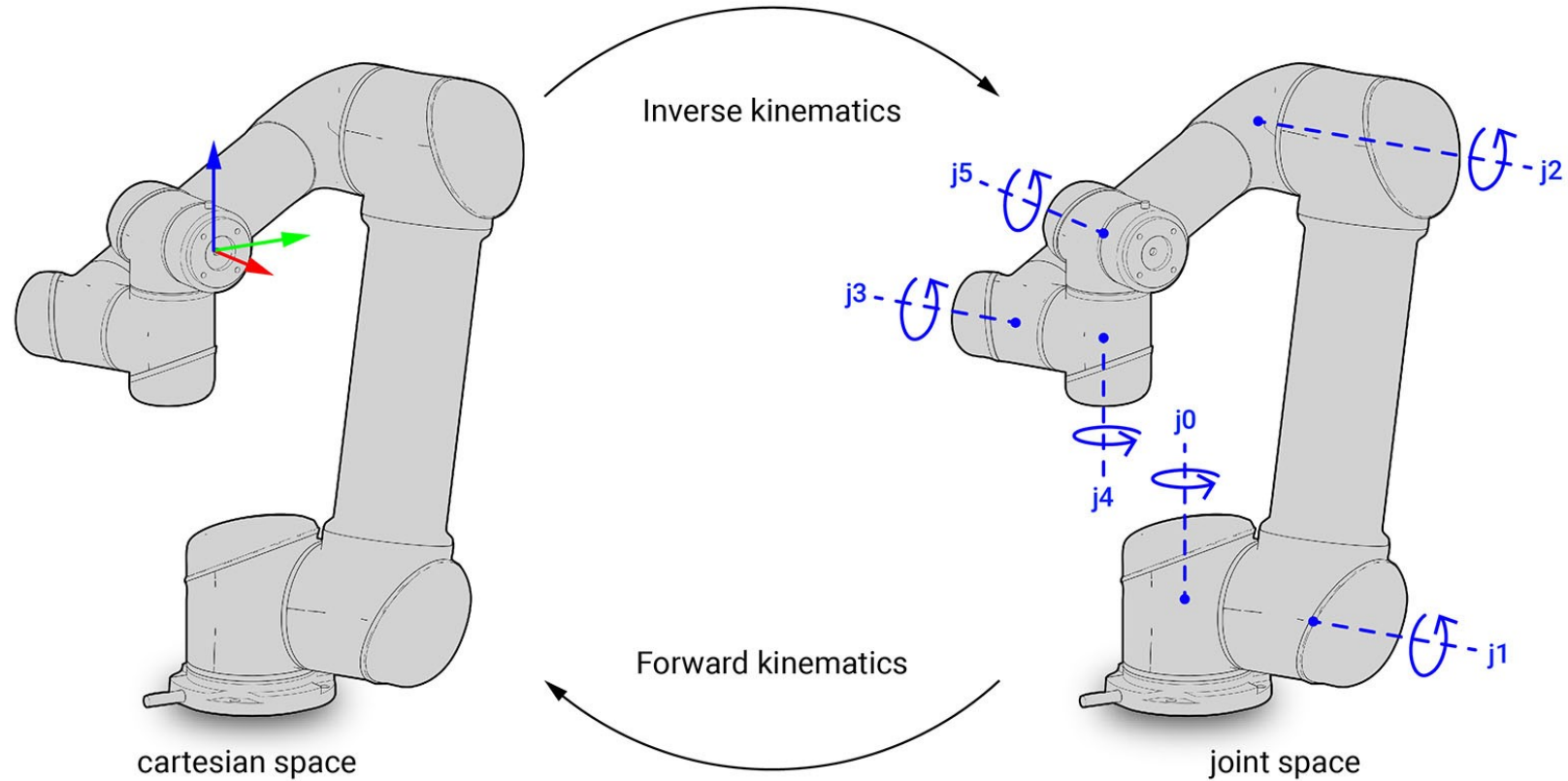
B



Motion Planning Definition

Motion planning is finding a sequence of configurations from start to goal such that every configuration along the way is valid.

Cartesian vs. Joint Space



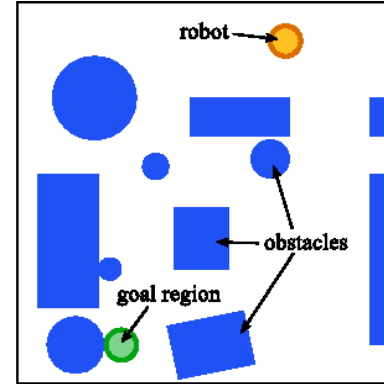
Constraints

» What makes a solution valid?

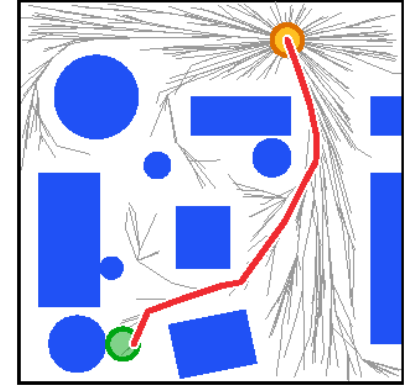
- » **Joint positions:** limits, obstacles, singularity, self-collisions
- » **Joint velocities:** limits
- » **Joint accelerations:** limits, forces
- » **Any More?** Tip: Think FK!

Sampling-based Planners

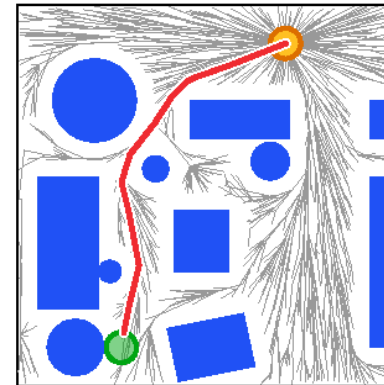
- » How to produce a plan?
 - » It's a search problem.
- » **OMPL: RRTConnect**
 - » Explores all the tree nodes till it finds a set of valid configurations that fulfill the constraints and reach the goal



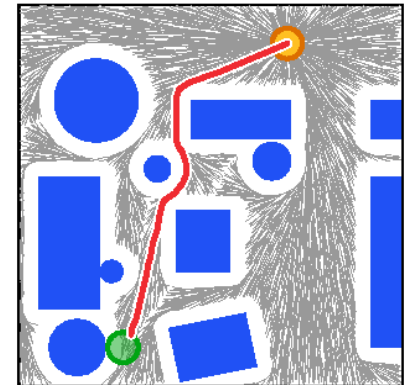
(a) The scenario



(b) 1 core



(c) 4 cores



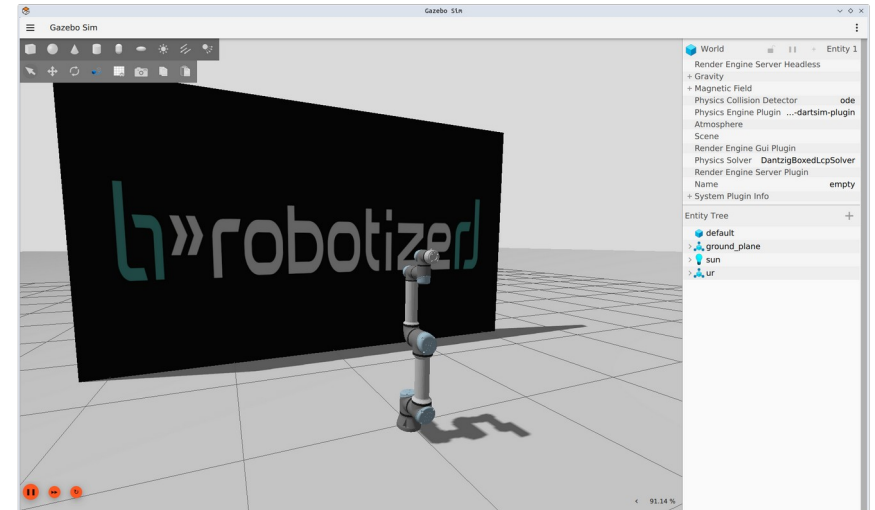
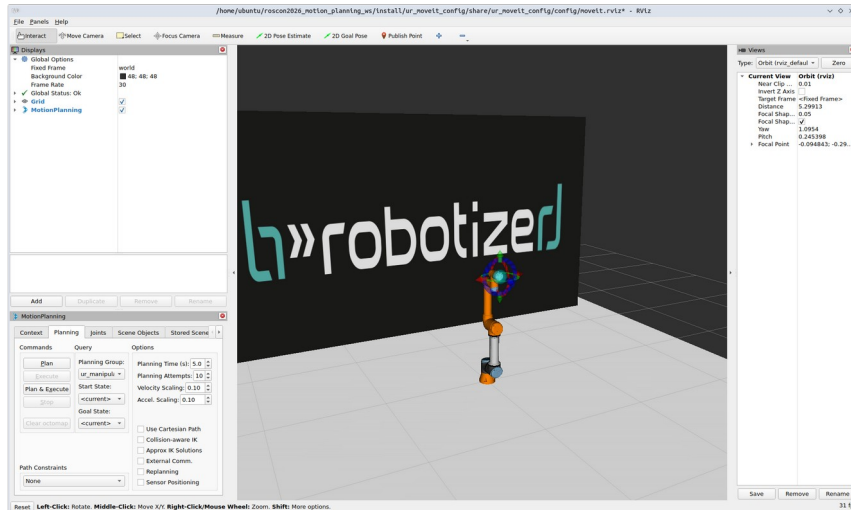
(d) 32 cores

UR Bringup

- » tf tree
- » controllers
- » How to send commands with JTC and forward

It's launch time!

- » `docker compose start`
- » `docker compose exec workshop bash`
- » `ros2 launch ur_simulation_gz ur_sim_moveit.launch.py`
`ur_type:=ur5e`

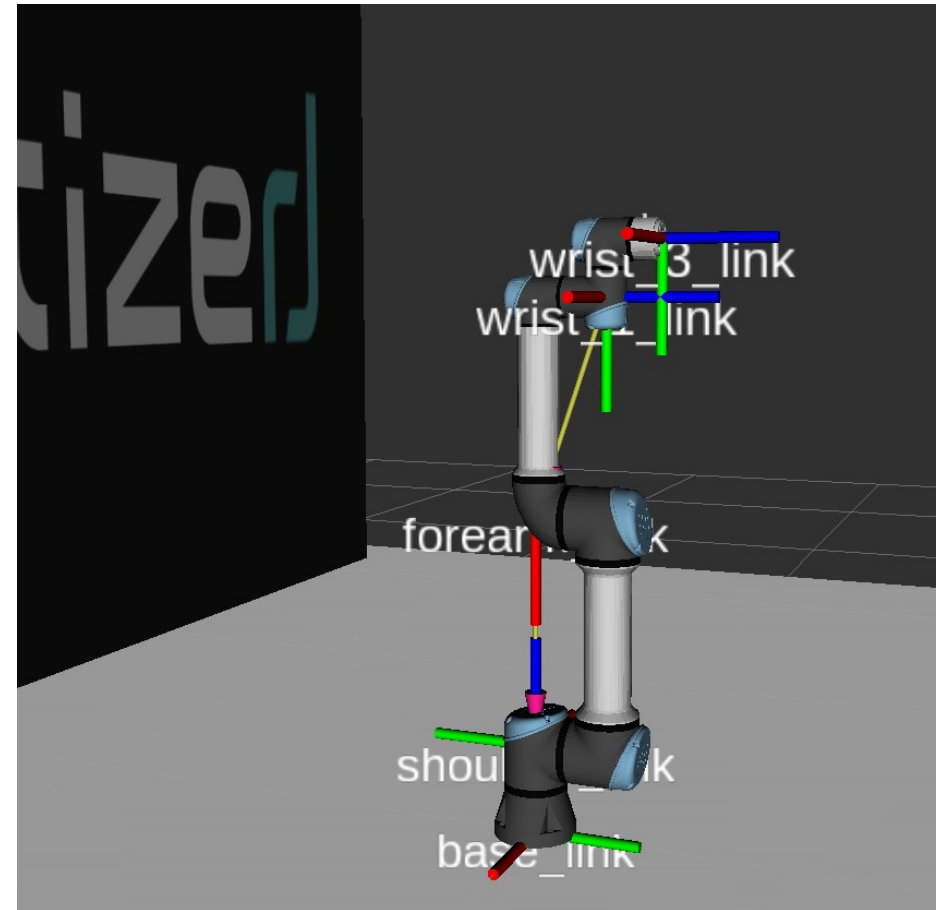


What We Just Launched

- » **robot_state_publisher**
- » **controller_manager**
- » **Controller spawners:**
 - » **Joint State Broadcaster**
 - » **Scaled Joint Trajectory Controller**
- » **move_group**
- » **Gazebo**
- » **Rviz**

The URDF is the Base!

- » Links and joints
 - the **TF tree**
- » `<ros2_control>` tags
 - which **interfaces** each joint exposes

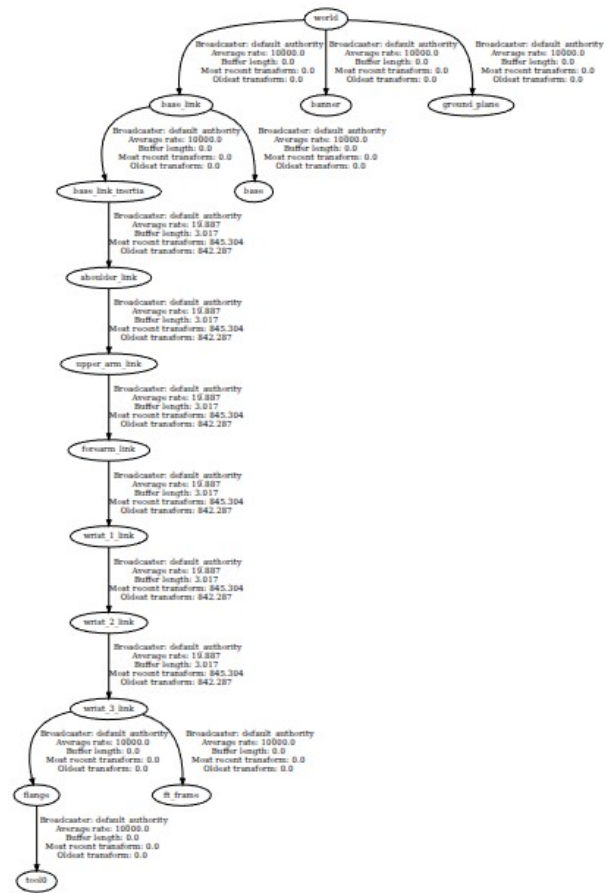


TF Tree: Forward Kinematics

» `ros2 run tf2_ros
tf2_echo base_link tool0`

» Joint values in
→ end-effector pose out

» `ros2 run tf2_tools
view_frames` for the whole tree



Command vs. State Interfaces

- » Every joint exposes **interfaces**, and there are two kinds
 - » **State**
 - » **Command**
- » ***sudo apt install ros-jazzy-ros2controlcli***
- » `ros2 control list_hardware_interfaces`
 - » `shoulder_pan_joint/position` as a **command** is a different resource from the same name as a **state**

Controllers Claim Interfaces Exclusively

- » Two controllers cannot own the same joint's position command
- » `ros2 control list_controllers`

```
joint_state_broadcaster          joint_state_broadcaster/JointStateBroadcaster  active
scaled_joint_trajectory_controller ur_controllers/ScaledJointTrajectoryController  active
```

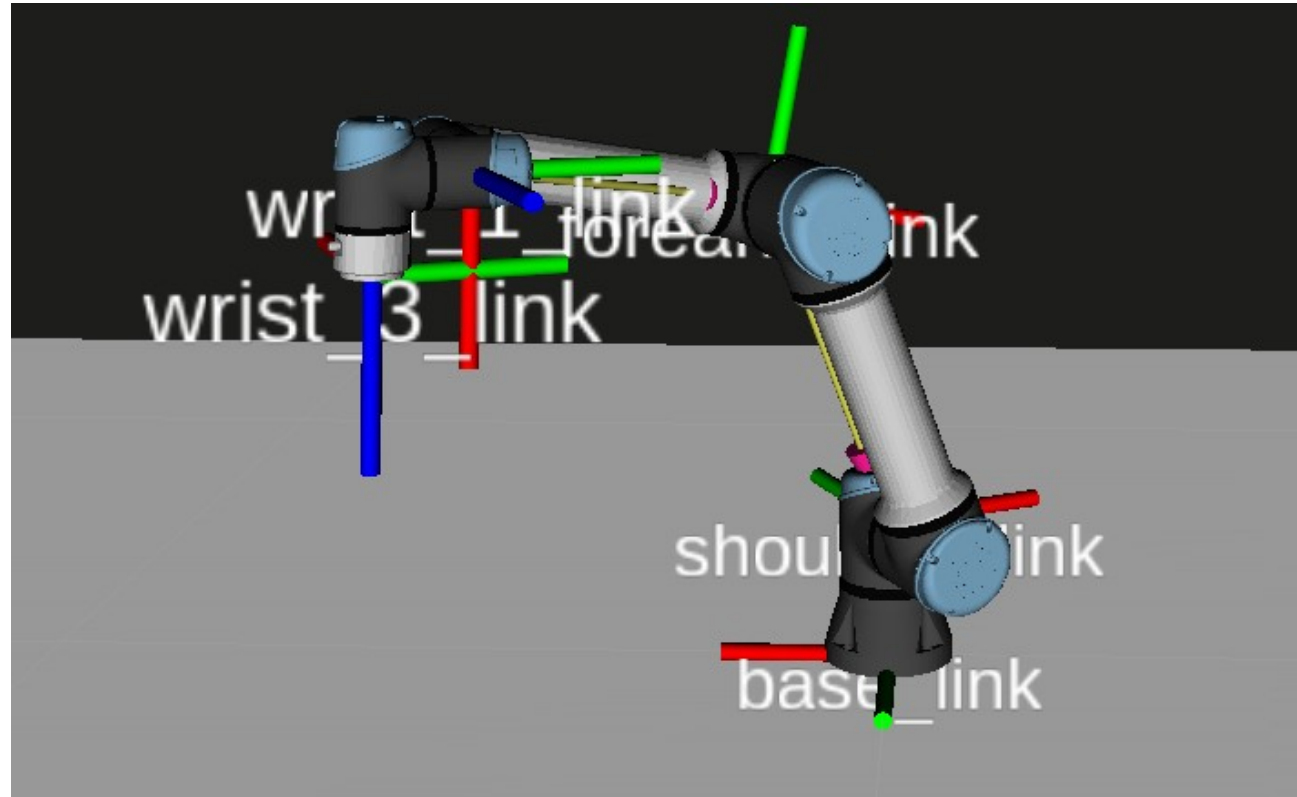
Commanding with the Joint Trajectory Controller

- » Message: trajectory_msgs/msg/JointTrajectory
- » Topic: /scaled_joint_trajectory_controller/joint_trajectory

```
# The header is used to specify the coordinate frame and the reference time for  
# the trajectory durations  
std_msgs/Header header  
  
# The names of the active joints in each trajectory point. These names are  
# ordered and must correspond to the values in each trajectory point.  
string[] joint_names  
  
# Array of trajectory points, which describe the positions, velocities,  
# accelerations and/or efforts of the joints at each time point.  
JointTrajectoryPoint[] points
```

JTC Interpolates for You

- » Change `time_from_start` & notice the difference!



The Forward Position Controller

- » Message: `std_msgs/msg/Float64MultiArray`
- » Topic:
`/forward_position_controller/commands`

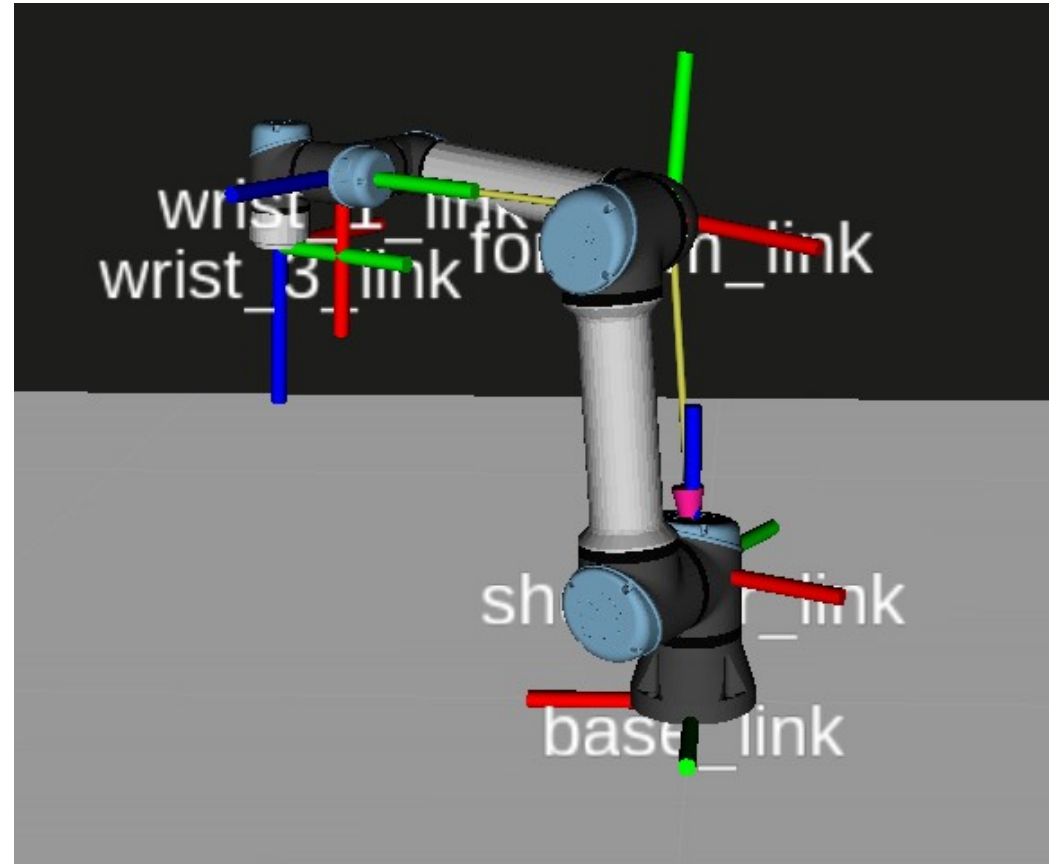
```
# This was originally provided as an example message.  
# It is deprecated as of Foxy  
# It is recommended to create your own semantically meaningful message.  
# However if you would like to continue using this please use the equivalent in example_msgs.  
  
# Please look at the MultiArrayLayout message definition for  
# documentation on all multiarrays.  
  
MultiArrayLayout  layout      # specification of data layout  
float64[]         data        # array of data
```

Why Didn't It Move?

- » `ros2 control list_controllers` → it is **inactive**
 - » The JTC holds the position command interfaces
- » Switch the controllers!

No Interpolation

- » Publish the same message again → **the robot jumps**
- » **Smoothness is now the commander's responsibility**



What Neither of Them Has

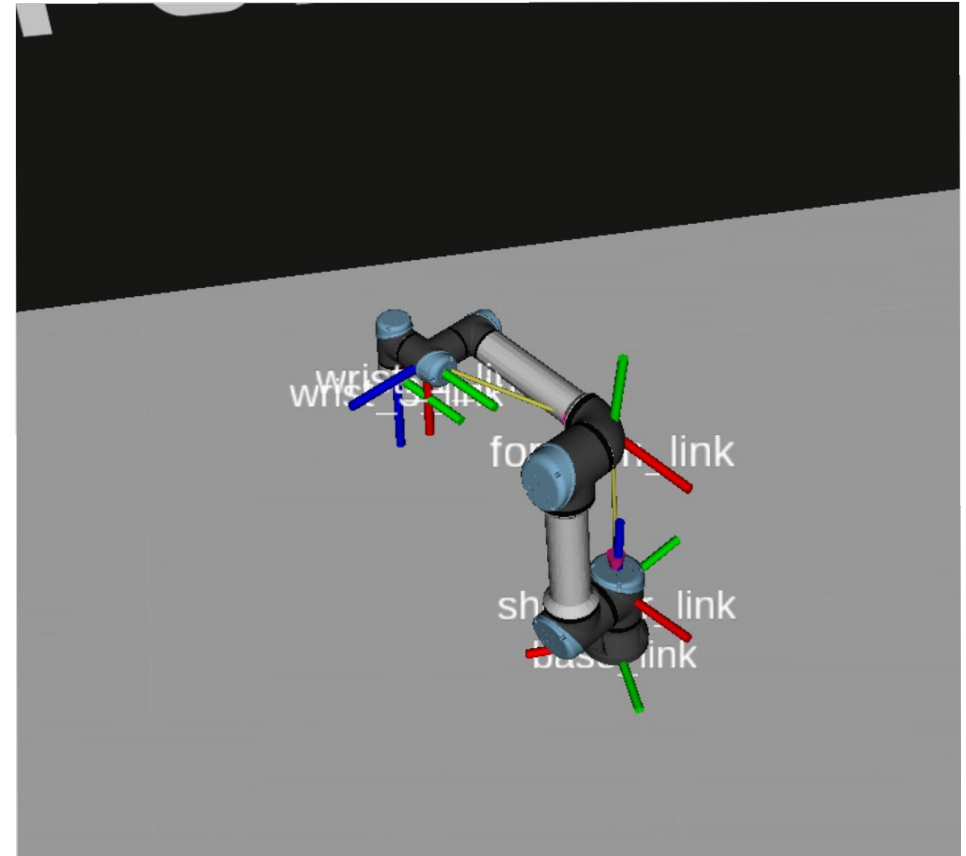
- » **No collision awareness:** neither knows the wall exists
- » Both assume **you already decided** what the joint values should be
- » **That gap is what MoveIt fills.**

Hands-on: Bringup & Direct Control

- » **1.** Current point: environment launched with forward controller.
- » **2.** `tf2_echo base_link tool0` & note the TCP position
- » **3.** Switch back to Joint Trajectory Controller
- » **4.** Try to construct a command to move the robot's **end-effector** to the right (Clockwise) with a **smooth motion**
- » **5.** `tf2_echo base_link tool0` again & verify the TCP position.

Hands-on: Bringup & Direct Control

» Checkout
section_2_checkpoint
for solution and
next steps!

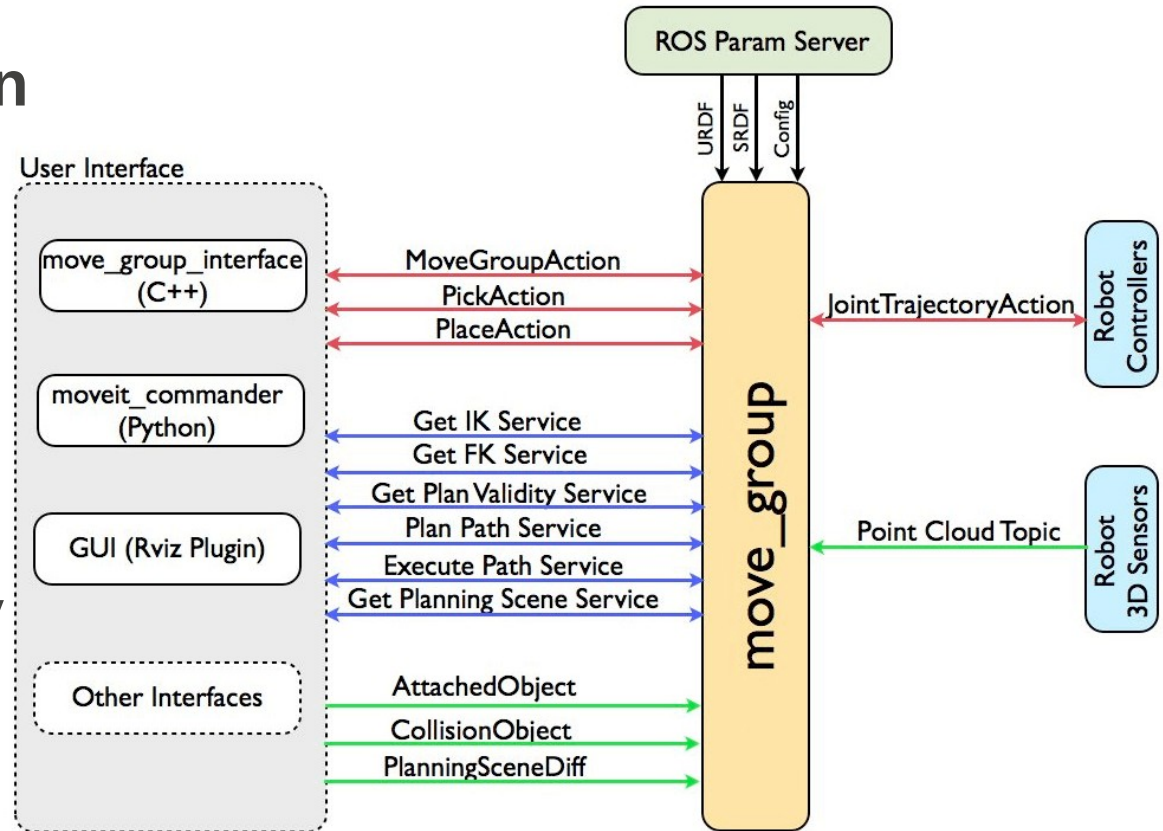


Moveit 2 Architecture

- » move group
- » Configurations

move_group: The Orchestrator

- » **move_group does not plan**
 - » It loads plugins, and asks them to do the work
- » It exposes **one action**: send a goal, get a trajectory back
- » Then it hands that trajectory to **ros2_control**



The Plugin types

- » **Planner:** OMPL, Pilz, CHOMP, STOMP
- » **Kinematics solver:** KDL, TRAC-IK, BioIK, IKFast
- » **Collision checker:** FCL
- » Each is **selected in a YAML file**
 - » Which is why configuration is the rest of this section

The Handoff to ros2_control

- » MoveIt plans. **It does not execute.**
- » It calls **the same controllers you commanded by hand.**

```
moveit_controllers.yaml

moveit_simple_controller_manager:
  controller_names:
    - scaled_joint_trajectory_controller
    - joint_trajectory_controller

scaled_joint_trajectory_controller:
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  default: true
  joints:
    - shoulder_pan_joint
    - shoulder_lift_joint
    - elbow_joint
    - wrist_1_joint
    - wrist_2_joint
    - wrist_3_joint
```

What Configures a MoveIt Setup

- » **.srdf**: planning groups, end effector, disabled collisions
- » **kinematics.yaml**: which IK solver, per group
- » **joint_limits.yaml**: velocity and acceleration caps
- » **ompl_planning.yaml** / pipeline configs: planner settings
- » **moveit_controllers.yaml**: how to reach
ros2_control

The SRDF

```
ur_macro.srdf.xacro

<robot xmlns:xacro="http://wiki.ros.org/xacro">
  <xacro:macro name="ur_srdf" params="">
    <group name="ur_manipulator">
      <chain base_link="base_link" tip_link="tool0" />
    </group>

    <disable_collisions link1="base_link" link2="base_link_inertia" reason="Adjacent" />
    <disable_collisions link1="base_link_inertia" link2="shoulder_link" reason="Adjacent" />
  /> <disable_collisions link1="tool0" link2="wrist_1_link" reason="Never" />
    <disable_collisions link1="tool0" link2="wrist_2_link" reason="Never" />
    <disable_collisions link1="tool0" link2="wrist_3_link" reason="Adjacent" />
    <disable_collisions link1="forearm_link" link2="upper_arm_link" reason="Adjacent" />
    <disable_collisions link1="forearm_link" link2="wrist_1_link" reason="Adjacent" />
    <disable_collisions link1="shoulder_link" link2="upper_arm_link" reason="Adjacent" />
    <disable_collisions link1="wrist_1_link" link2="wrist_2_link" reason="Adjacent" />
    <disable_collisions link1="wrist_1_link" link2="wrist_3_link" reason="Never" />
    <disable_collisions link1="wrist_2_link" link2="wrist_3_link" reason="Adjacent" />

  </xacro:macro>
</robot>
```

kinematics.yaml

- » Selects the **IK solver plugin, per planning group**
- » **KDL**: Numerical, general-purpose, for any chain
- » **kinematics_solver_timeout**: how long before it gives up

```
kinematics.yaml

ur_manipulator:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.00500000000000000001
  kinematics_solver_timeout: 0.00500000000000000001
```

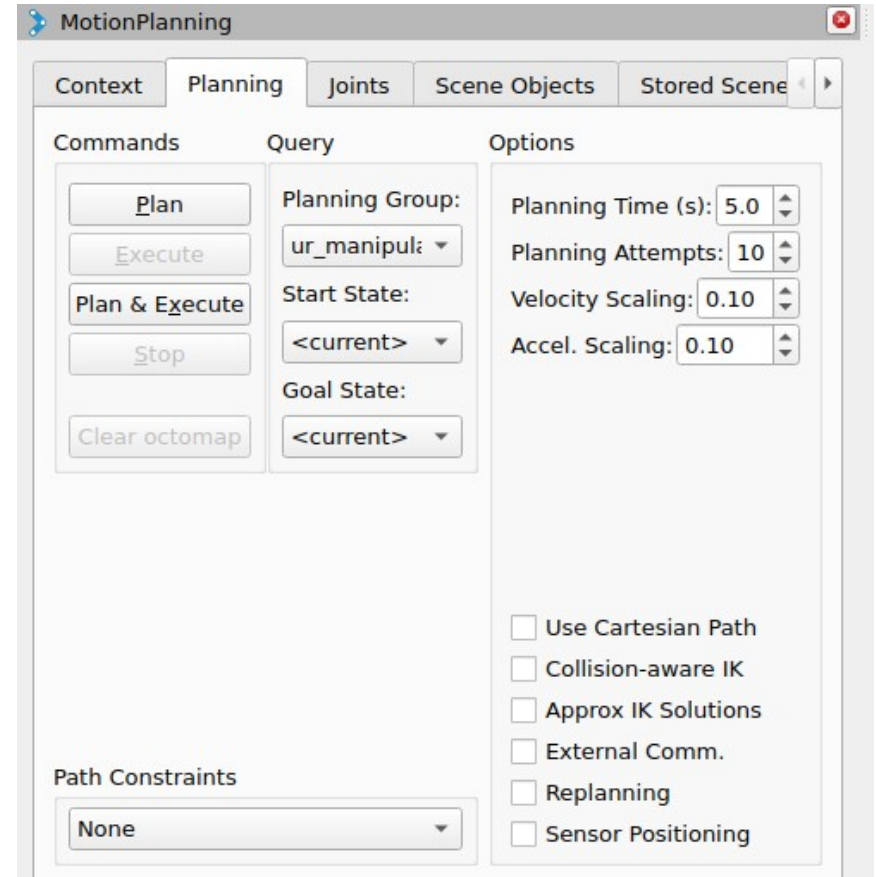
joint_limits.yaml

```
joint_limits.yaml

joint_limits:
  shoulder_pan_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
  shoulder_lift_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
  elbow_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
  wrist_1_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
  wrist_2_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
  wrist_3_joint:
    has_acceleration_limits: true
    max_acceleration: 5.0
```

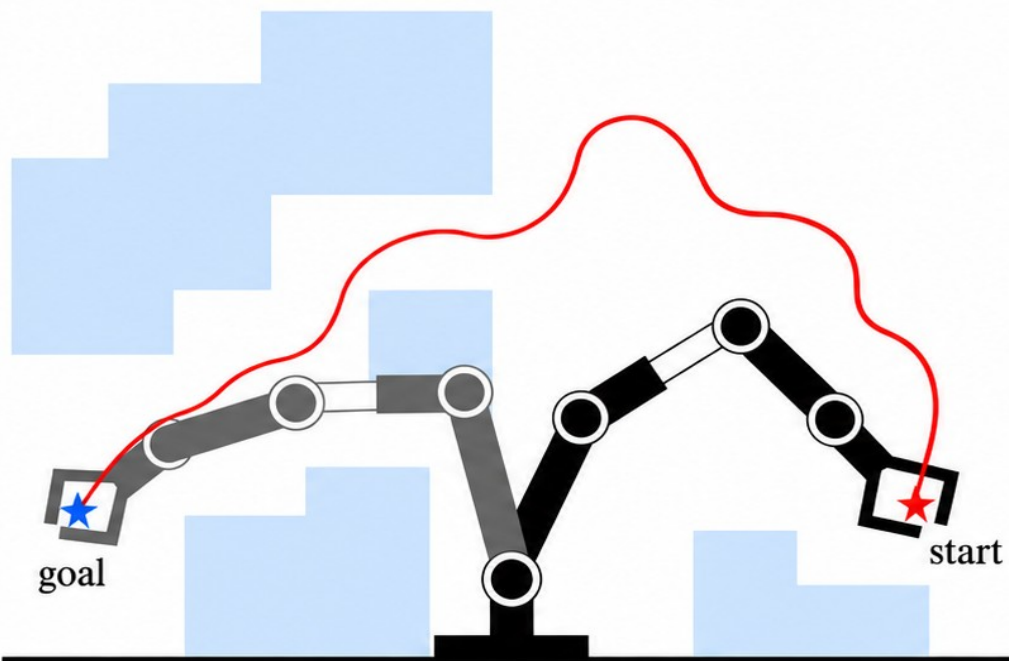
The RViz MotionPlanning Panel

- » **Planning group** dropdown: the groups from your SRDF
- » Interactive marker
- » **Plan** then **Execute** are separate buttons for a reason
 - » Plan is a request. Execute is the `ros2_control` handoff.
- » Planned path can be replayed without touching the robot

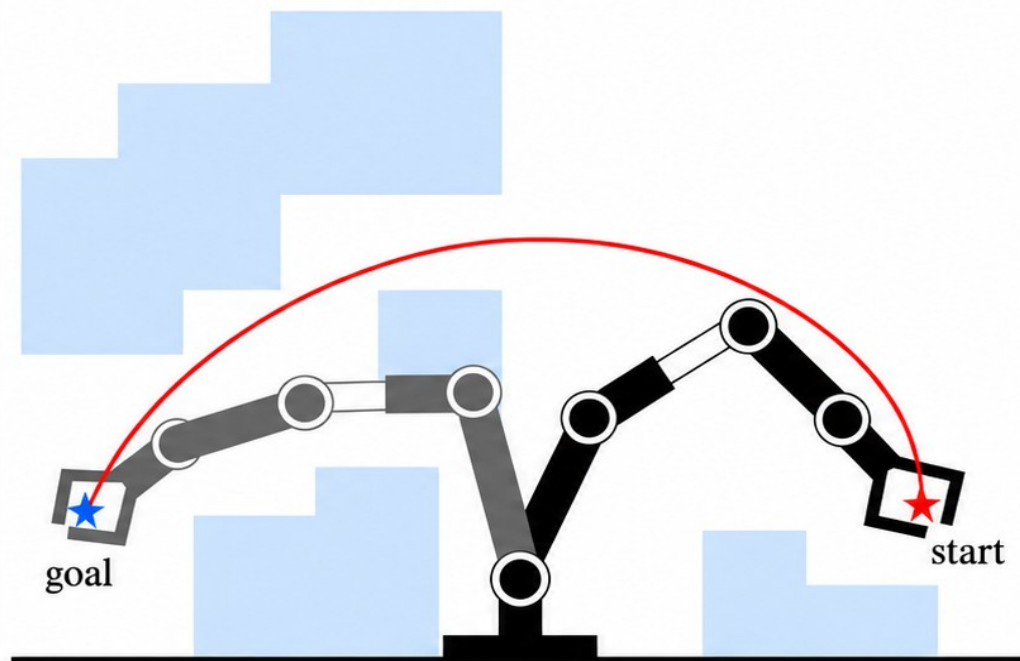


Sampling vs. Deterministic Motion Generation

OMPL



Pilz



Hands-on: Build a Planning Group

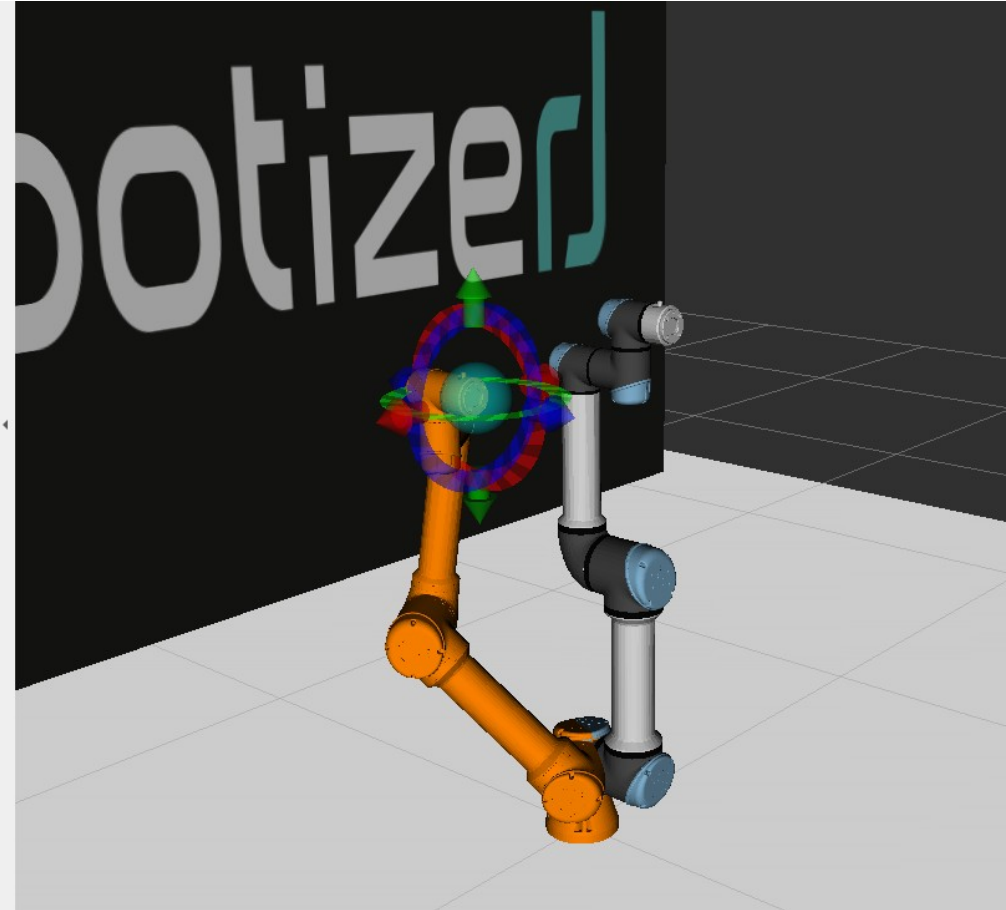
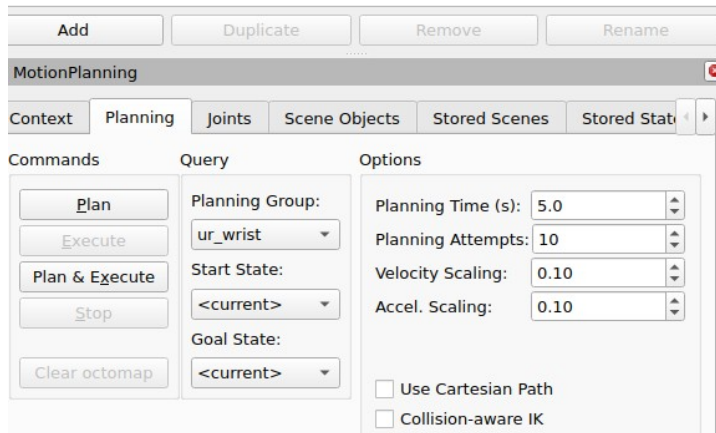
- » **1.** Open the SRDF. Find the existing `ur_manipulator` group
- » **2.** Add a new group: chain from `base_link` to `wrist_3_link`
- » **3.** Rebuild & re-launch: try to make it work!
- » **4.** Select your group in the RViz dropdown and plan to a pose

Hands-on: Swap the Planner

- » **5.** Plan the same goal twice with OMPL. Compare the paths
- » **6.** Switch the pipeline to **Pilz PTP**. Plan twice more. Notice a difference?

Hands-on: Checkpoint

» Checkout **section_3_checkpoint** for the solution and next steps!



It's Coffee Time!

Planning Scene and Collision

- » Scene Representation
- » Scene manipulations: add, remove, attach, detach
- » Hands-on

The Planning Scene

- » Robot state
 - » Attached objects
- » World
 - » collision objects + octomap
- » Every plan request is checked against **this** snapshot

```
# name of planning scene
string name

# full robot state
RobotState robot_state

# The name of the robot model this scene is for
string robot_model_name

#additional frames for duplicating tf (with respect to the planning frame)
geometry_msgs/TransformStamped[] fixed_frame_transforms

#full allowed collision matrix
AllowedCollisionMatrix allowed_collision_matrix

# all link paddings
LinkPadding[] link_padding

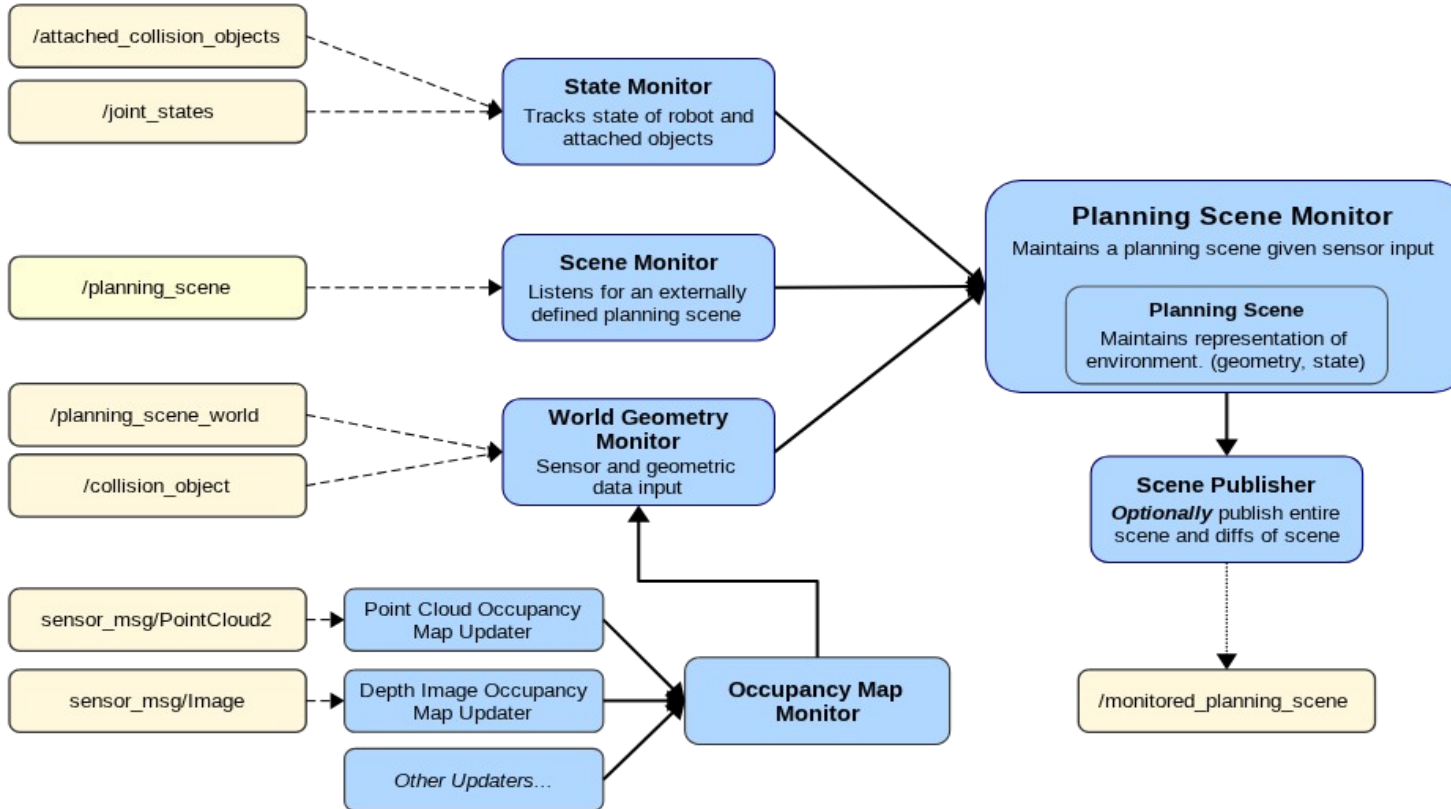
# all link scales
LinkScale[] link_scale

# Attached objects, collision objects, even the octomap or collision map can have
# colors associated to them. This array specifies them.
ObjectColor[] object_colors

# the collision map
PlanningSceneWorld world

# Flag indicating whether this scene is to be interpreted as a diff with respect to some other
bool is_diff
```

Planning Scene Monitor



What Collision Checking Checks

- » **Self-collision:** Robot against itself
- » Robot against the world
- » Attached objects against everything (incl. Octomap)
- » Checked at **every configuration along the path**
 - » Not just at the start and the goal

The Allowed Collision Matrix

- » A table of **link pairs**:
check this pair, or don't
- » Adjacent links should be disabled
- » 'Allowed' means 'I know, don't tell me'

	spacecraft_link0	spacecraft_link1	spacecraft_link2	spacecraft_link3	spacecraft_link4	spacecraft_link5	spacecraft_link6	spacecraft_link7	spacecraft_hand	spacecraft_leftfinger	spacecraft_rightfinger
spacecraft_link0		✓	✓	✓	✓	✓	✓	✓	□	□	□
spacecraft_link1	✓		✓	✓	✓	✓	✓	□	□	✓	✓
spacecraft_link2	✓	✓		✓	✓	✓	✓	□	□	□	□
spacecraft_link3	✓	✓	✓		✓	✓	✓	□	✓	✓	✓
spacecraft_link4	✓	✓	✓	✓		✓	□	✓	✓	✓	✓
spacecraft_link5	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓
spacecraft_link6	✓	✓	✓	✓	□	✓		✓	✓	✓	✓
spacecraft_link7	✓	□	□	□	✓	✓	✓		✓	✓	✓
spacecraft_hand	□	□	□	✓	✓	✓	✓	✓		✓	✓
spacecraft_leftfinger	□	✓	□	✓	✓	✓	✓	✓	✓		✓
spacecraft_rightfinger	□	✓	□	✓	✓	✓	✓	✓	✓	✓	

Planning Scene Actions:

Collision Objects

» Operations:

» ADD

» REMOVE

» MOVE

» Attach

» Detach

» Objects persist in the scene until you remove them

```
GetPlanningScene.srv

# Get parts of the planning scene that are of interest
# All scene components are returned if none are specified
PlanningSceneComponents components
---
PlanningScene scene

ApplyPlanningScene.srv

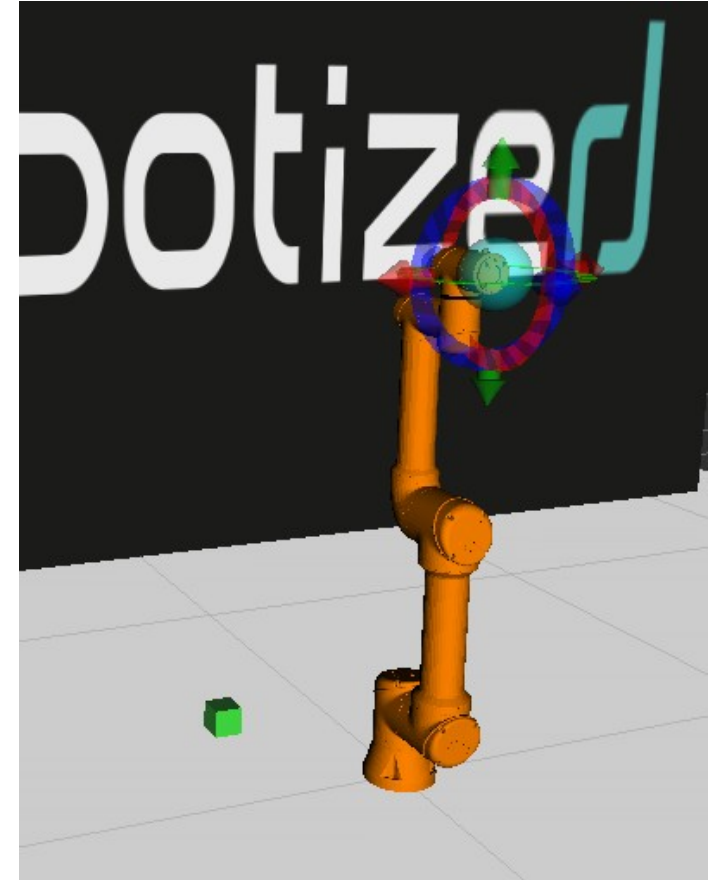
PlanningScene scene
---
bool success
```

Actions: Add Objects



Add Object CLI command

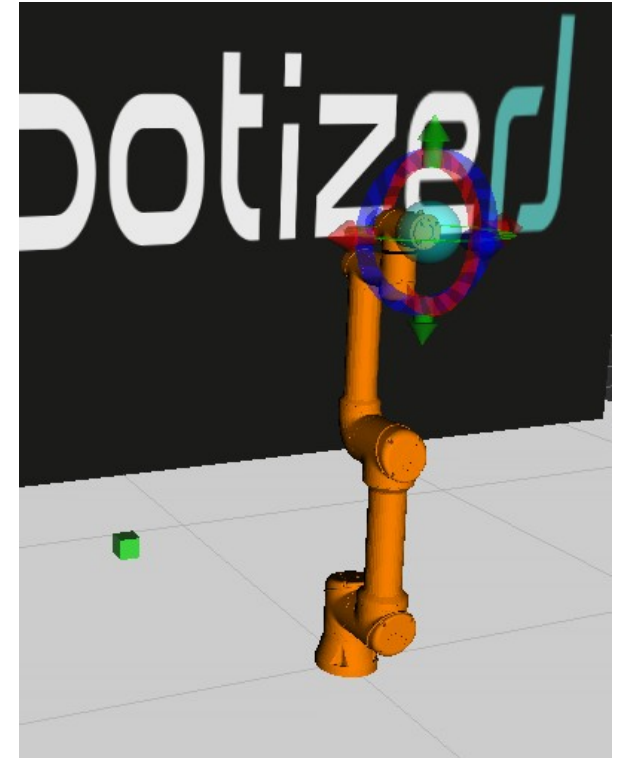
```
ros2 service call /apply_planning_scene
moveit_msgs/srv/ApplyPlanningScene "{
  scene: {
    is_diff: true,
    world: {
      collision_objects: [{
        header: {frame_id: base_link},
        id: cube,
        operation: 0,
        primitives: [{type: 1, dimensions: [0.05, 0.05, 0.05]}],
        primitive_poses: [{position: {x: 0.5, y: 0.2, z: 0.3},
orientation: {w: 1.0}}]
      }]
    }
  }
}"
```



Actions: Move Objects

Move Object CLI command

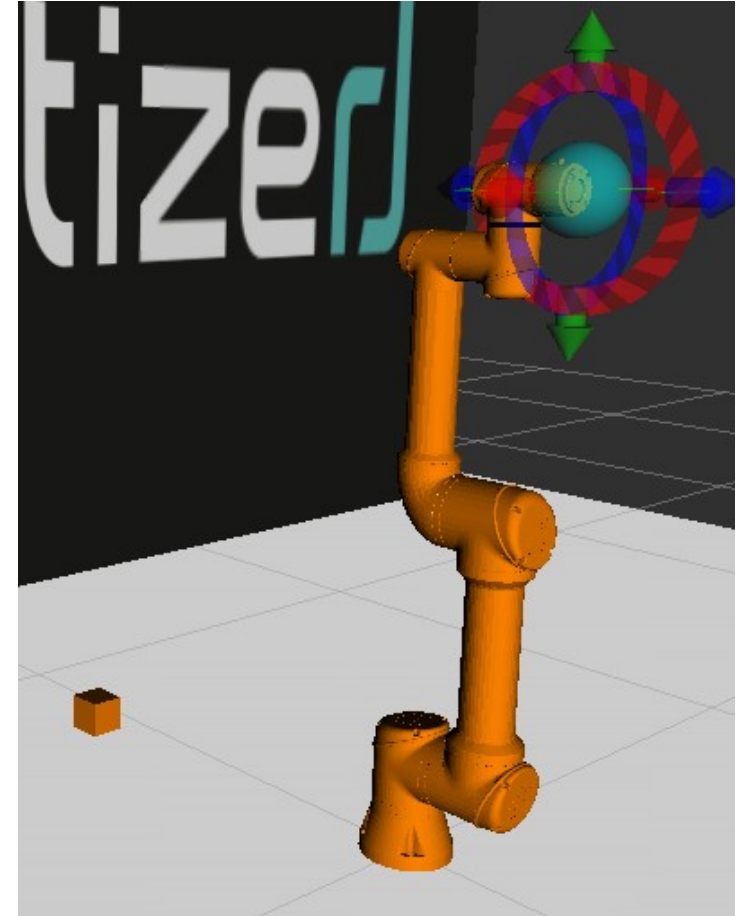
```
ros2 service call /apply_planning_scene
moveit_msgs/srv/ApplyPlanningScene "{
  scene: {
    is_diff: true,
    world: {
      collision_objects: [{
        header: {frame_id: base_link},
        id: cube,
        operation: 0,
        primitives: [{type: 1, dimensions: [0.05, 0.05, 0.05]}],
        primitive_poses: [{position: {x: 0.5, y: -0.2, z: 0.3},
orientation: {w: 1.0}}]
      }]
    }
  }
}"
```



» Note: same add command with same ID and different pose!

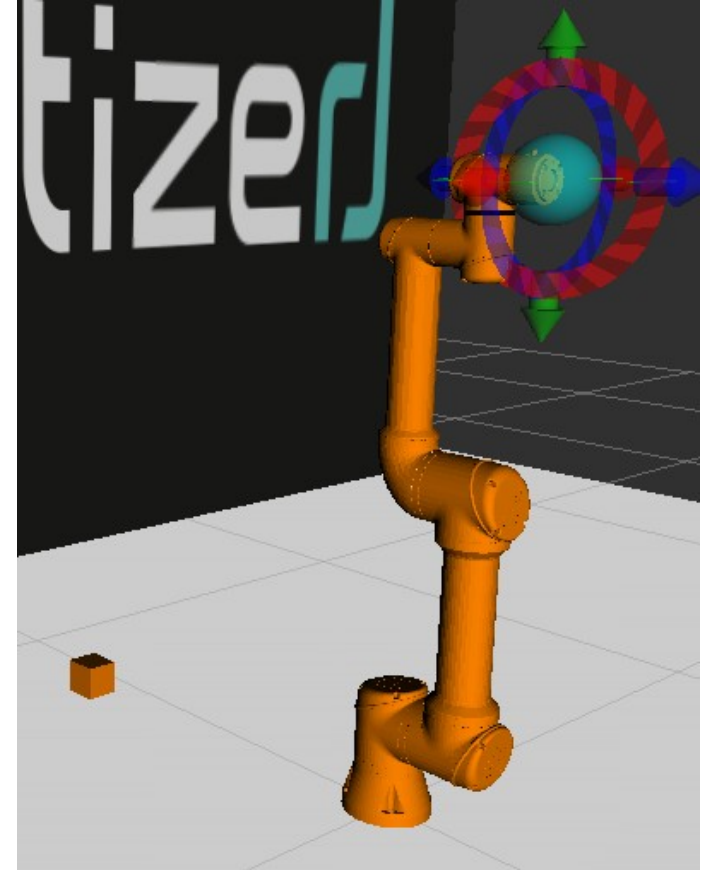
Attaching an Object

- » The object **leaves the world and joins the robot**
- » It moves with the link it is attached to
- » Its volume is now part of what gets collision-checked
- » **touch_links:** links allowed to contact it
- » Notice the object color change.
- » Try to plan & move now with Rviz tool!



Hands-on: Checkpoint

» Checkout
section_4_preparation
for the solution and next steps!



Hands-on: From CLI to Code

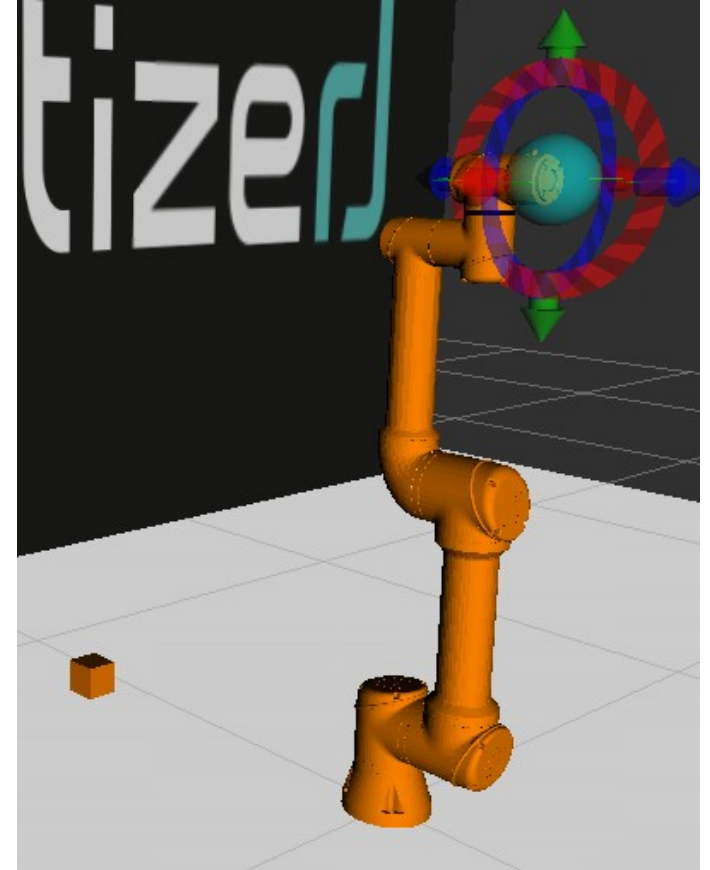
- » You have just done all of this from the terminal. Now do the same from **rclpy**.
- » Look it up. The **Movelit and moveit_msgs docs** are the exercise.
- » Two files:
 - » **utils.py**: the helpers
 - » **workshop_scenario.py**: a node that calls them

Hands-on: Hands-on: What to Build

- » **utils.py** three functions:
 - » **add_object(...)**
 - » **attach_object(...)**
 - » **detach_object(...)**
- » **workshop_scenario.py** reads as function calls to replicate what we just did with CLI:
 - » **add_object(...)**
 - » **attach_object(...)**
- » Use the named constants: `CollisionObject.ADD`, `.REMOVE`

Hands-on: Checkpoint

» Checkout
section_4_checkpoint
for the solution and next steps!



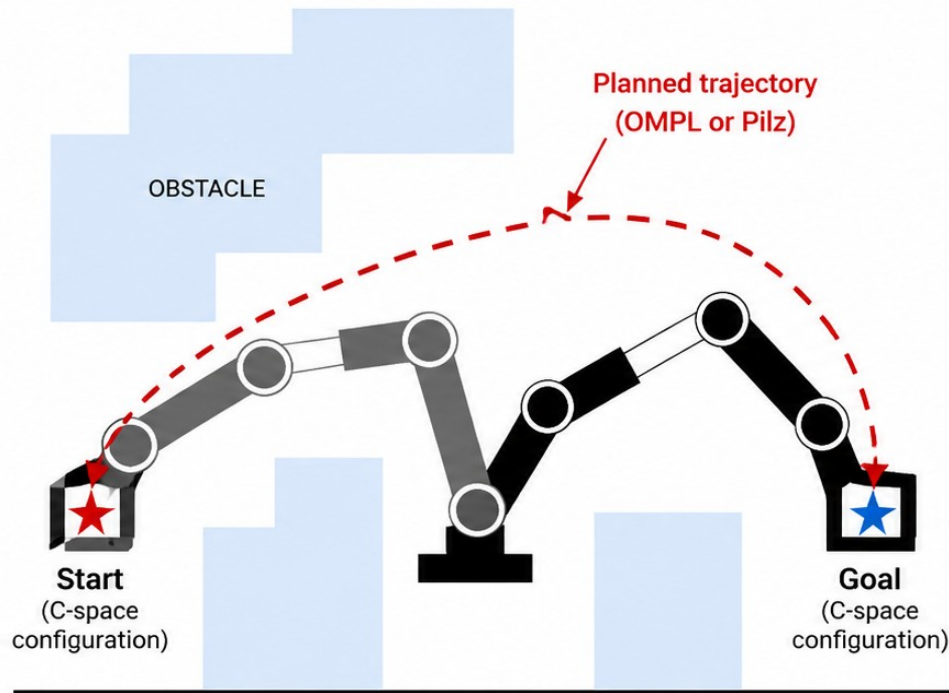
Moveit Servo

- » What is it and its use-cases
- » How does it work
- » Hands-on

Two Ways to Move a Robot

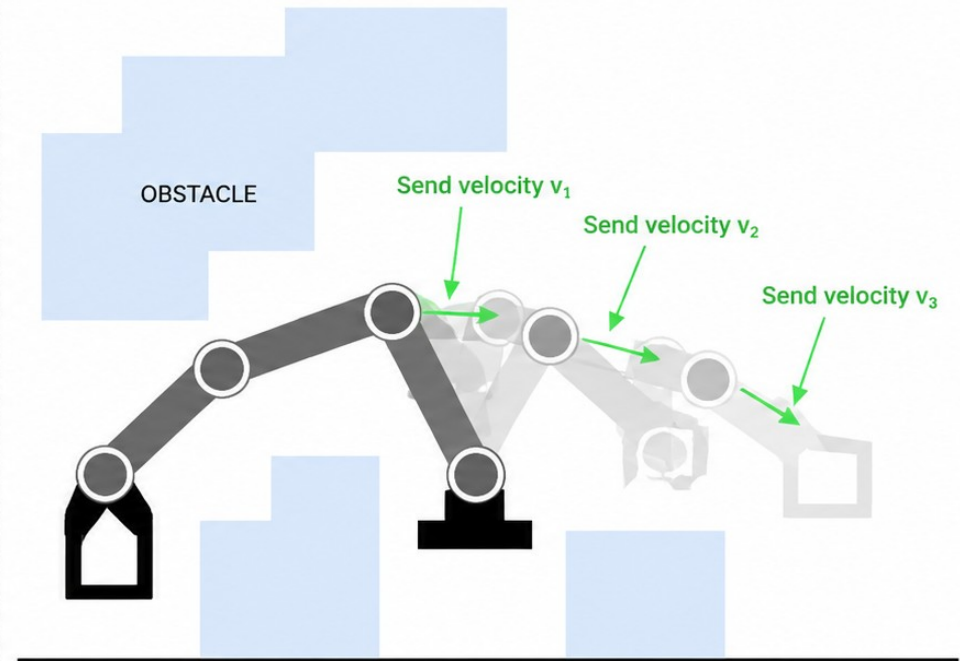
Plan and execute: everything so far today

Give a goal, wait, get a whole trajectory, run it.



Streaming: Servo

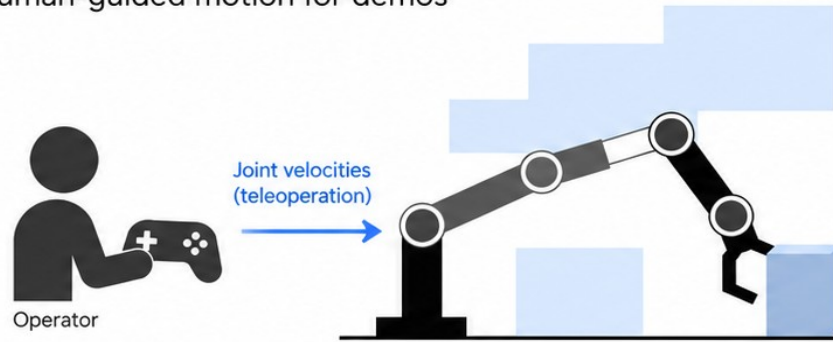
Send a velocity. The robot moves now. Send another.
No goal. No search. No trajectory.



Moveit Servo usecases

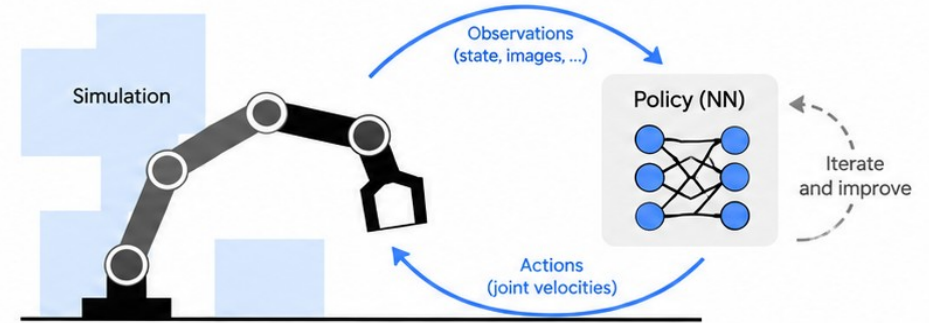
Collect demonstrations

Human-guided motion for demos



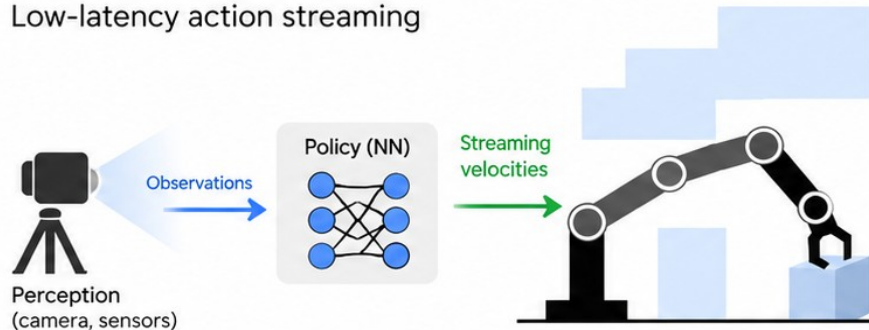
Train policies in simulation

Fast rollout and policy training



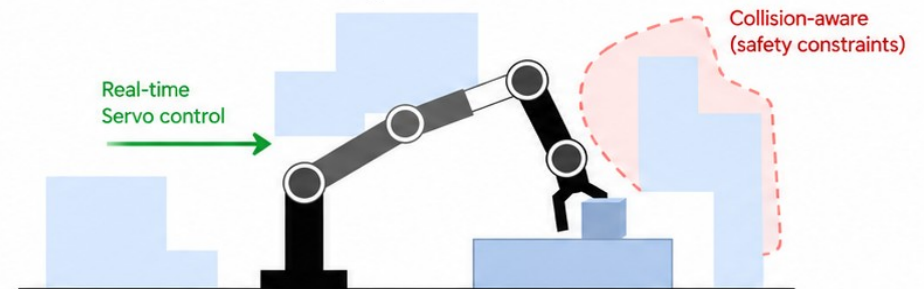
Run learned policies

Low-latency action streaming



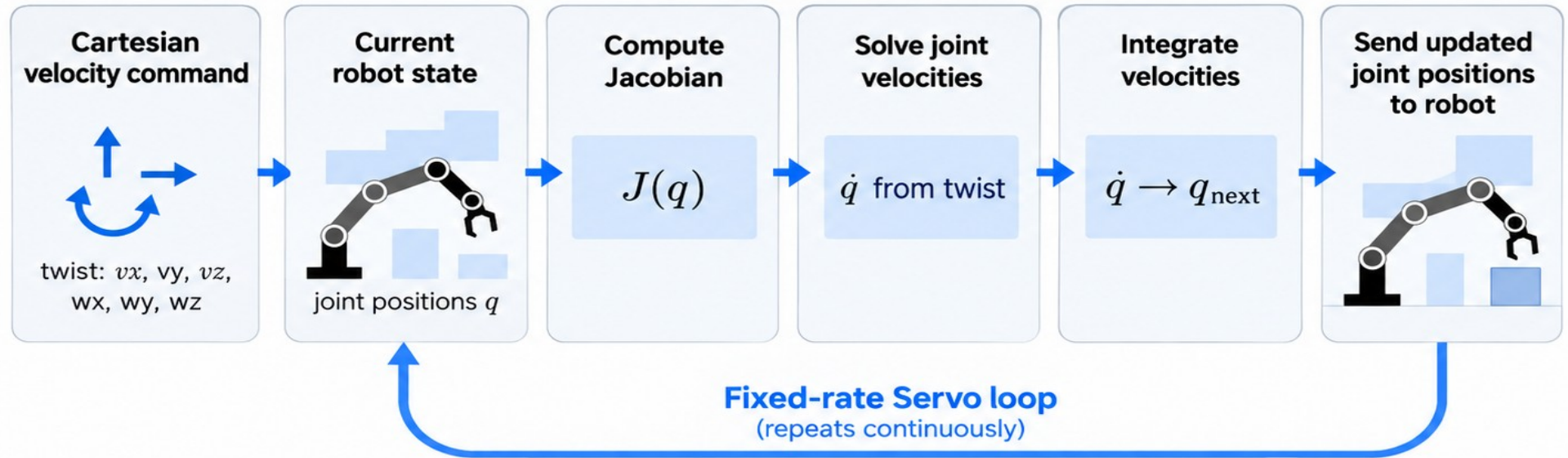
Fine-tune on the robot

Safe online adaptation



What Servo Actually Does

» Velocity-level IK, running continuously



Where the Output Goes

- » Servo publishes to
`/forward_position_controller/commands`
- » As a `Float64MultiArray`
- » Everything above it is what Servo adds

Talking to Servo

- » **Mode 0: /servo_node/delta_joint_cmds, JointJog**
 - » Per-joint velocity, no IK involved
- » **Mode 1: /servo_node/delta_twist_cmds, TwistStamped**
 - » Cartesian velocity of the end effector
- » **Mode 2: /servo_node/pose_target_cmds, PoseStamped**
 - » Target pose for the end effector; Servo drives toward it

Try it yourself!

- » 1. Launch the environment
- » 2. Get the robot out of singularity pose
- » 3. Launch forward command controller
- » 4. Switch to forward command controller
- » 5. Switch moveit servo to twist mode
- » 6. Send a twist command

Hands-on: Keyboard Teleop

- » From the last state we reached, we can now send out **Twist** commands for the robot to obey.
- » **Now:** Run `teleop_twist_keyboard`
- » Debugging time! Why doesn't it work? Can you fix it?
- » Checkout **section_5_checkpoint** for the solution and next steps!

Thank You:)

» Special Thanks:

- » PickNik
- » UR

» Questions?

- »  yara.shahin@b-robotized.com
- » [https://github.com/b-robotized/
roscon2026_motion_planning_workshop](https://github.com/b-robotized/roscon2026_motion_planning_workshop)

Join the conversation

Homework!

Hands-on Project: Design your own Pick & Place

- » How to use the action interfaces
- » Hands-on

Your First Planning Request

- » Send action to: `/move_action`
- » As `moveit_msgs/action/MoveGroup`
- » `MotionPlanRequest`
 - » Start state
 - » **Goal:** joint values, or a pose
 - » `pipeline_id`
 - » `planner_id`
 - » `group_name`
 - » . . .

```
# Motion planning request to pass to planner
MotionPlanRequest request

# Planning options
PlanningOptions planning_options

---

# An error code reflecting what went wrong
MoveItErrorCodes error_code

# The full starting state of the robot at the start of the trajectory
moveit_msgs/RobotState trajectory_start

# The trajectory that moved group produced for execution
moveit_msgs/RobotTrajectory planned_trajectory

# The trace of the trajectory recorded during execution
moveit_msgs/RobotTrajectory executed_trajectory

# The amount of time it took to complete the motion plan
float64 planning_time

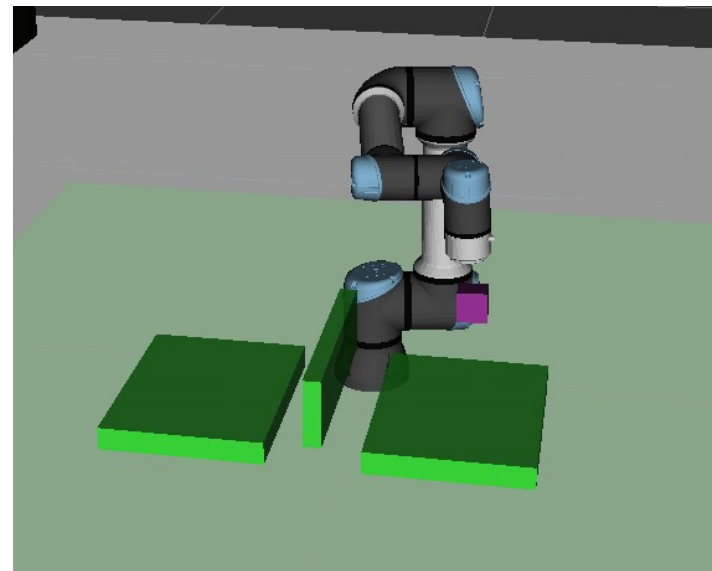
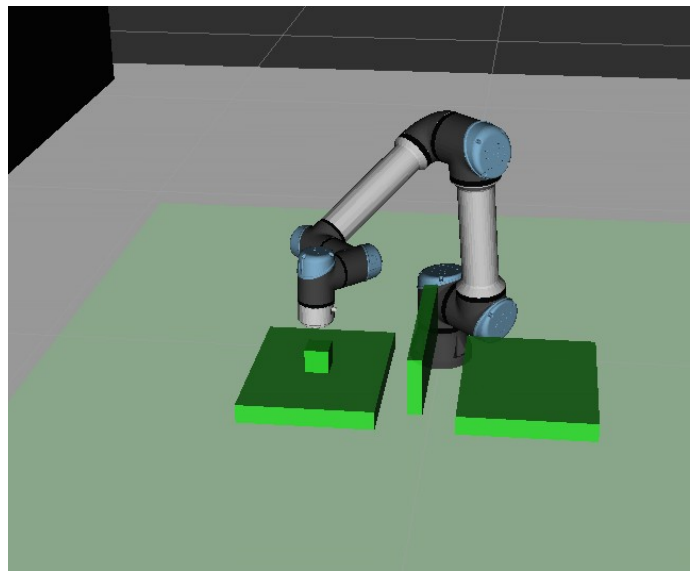
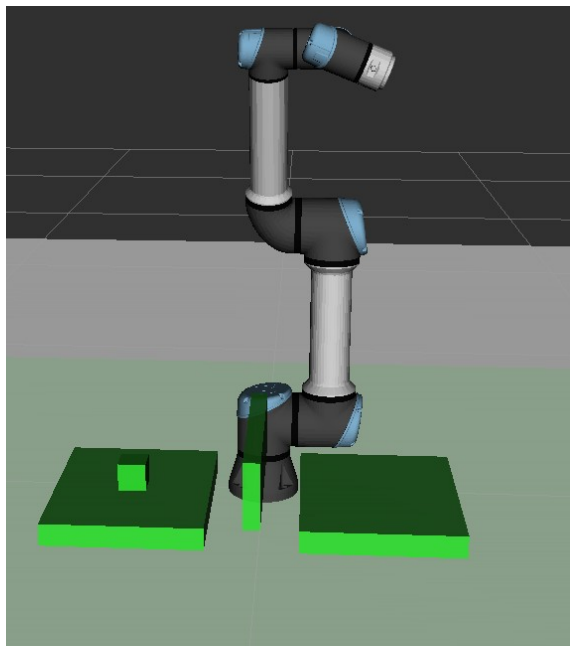
---

# The internal state that the move group action currently is in
string state
```

Hands-on: Capstone P&P Project

- » Construct this scene using your earlier `utils.py`
 - » **Two tables**, separated by a **dividing wall**
 - » A **small cube** sitting on table A
- » Using `rc1py`, make this scenario:
 - » Robot go to cube on table A, attach it
 - » Robot go to table B, detach it.
 - » Robot go home.

The target demo



Hands-on: Capstone P&P Project

» Checkout
section_6_checkpoint
for the solution and next steps!